How to sort an array in ascending order :

* Java has provided a predefined class called **Arrays available in java.util package.**
* Arrays which is utility class has provided following static methods : [All the methods are static only]

1) public static void sort(int [] arr) : It will sort the given integer array in ascending order.
   Example :
   int []arr = {12,9,1,5,9,15};
   Arrays.sort(arr); //sort the array in ascending order

```
SortInt.java
------------
//Sorting the array

void main()
{
  int []arr = {9,1,8,2,7,5};

   Arrays.sort(arr);

   for(int x : arr)
     {
          IO.print(x+" ");
     }
   IO.println();
}
```

Note : In compact source file approach, All the classes which are available in **java.base module** will be automatically imported so we need not to write any import statement.

```
import java.util.Arrays;

class Loop
{
     void main()
     {
       int []arr = {9,1,8,2,7,5};

          Arrays.sort(arr);

          for(int x : arr)
          {
               IO.print(x+" ");
          }
          IO.println();
     }
}
```

It is not a comact source file approach so, we need to import Arrays class

2) public static void sort(Object []arr) : It will sort the array elements in ascending order for reference data type

```
SortString.java
------------------
//Ascending order

void main()
{
     String []fruits = {"Orange", "Mango", "Apple", "Grapes"};

   Arrays.sort(fruits);

   for(String fruit : fruits)
     {
          IO.println(fruit);
     }

}
```

Note : No, We cannot sort the array if array will take different kinds of values using Object class

---

All possible combinations of methods:

The following cases are describing all the possible combinations of methods.

Case 1:

Method with no return type and no parameter.
Example :
```
public void accept()
{

}
```

Case 2:

Method with return type and no parameter.
Example :
```
public int accept()
{
  return 0;
}
```

Case 3:

Method with no return type and with parameter.
Example :
```
public void accept(int x, int y)
{

}
```

Case 4:

Method with return type and with parameter.
Example :
```
public int accept(int x, int y)
{
  return x+y;
}
```

Calling OR Invoking a method using IO.println() OR System.out.println() :

* We can **only** call OR invoke those methods whose return type is **not void from IO.println() OR System .out.println().**

* If any method return type is void then we cannot call the method from IO.println() OR System.out.println().

```
MethodCall1.java
-------------------
void main()
{
 int a = Integer.parseInt(IO.readln("Enter the value of a :"));
 int b = Integer.parseInt(IO.readln("Enter the value of b :"));
 IO.println("Sum is :"+sum(a,b));

}

int sum(int x, int y)
{
 return x + y;
}
```

```
MethodCall2.java
-------------------
void main()
{
 int a = Integer.parseInt(IO.readln("Enter the value of a :"));
 int b = Integer.parseInt(IO.readln("Enter the value of b :"));
 IO.println("Sub is :"+sub(a,b)); //compilation error because sub method return type is void

}
void sub(int x, int y)
{
     IO.println("Sub is :"+(x-y));
}
```

Numeric Logical Programs :

Algorithm OR Universal Rule :

Step 1 :
Take a number from the user

Step 2 :
If required, stored the original number in a temporary variable

Step 3 :
Declare all the required variables [Example sum = 0, prod =1]

Step 4 :
Repeat the number until number becomes zero using loop

Step 5 :
Extract the last digit of the number using % [num % 10]

Step 6 :
Write OR process the logic

Step 7 :
Reduce the number OR cut the last digit by / operator [num = num /10]

Step 8 :
Print the result outside of the loop

```
CountDigits.java
--------------------
/* WAP to count the digits in a given number
   92  = 2
   145 = 3
   1897 = 4
   12345 = 5
   00000 = 1
*/

void main()
{
     int number = Integer.parseInt(IO.readln("Enter a number :"));
     IO.println("Total digits in "+number+" are :"+countDigits(number));
}

int countDigits(int num)
{
 if(num==0)
     {
       return 1;
     }

 int count = 0; //3

 while(num!=0)
 {
    num = num /10;
      count++;
 }
 return count;
}
```

```
SumOfDigits.java
-------------------
/*WAP to count the sum of the digits in a given number.
  45 = 9
  123 = 6
  444 = 12
  1278 = 18
*/

void main()
{
     int number = Integer.parseInt(IO.readln("Enter a number :"));
     IO.println("Sum of digits are :"+sumOfDigits(number));
}

int sumOfDigits(int num)  //num = 0
{
   int sum = 0; //9

   while(num!=0)
   {
     int digit = num % 10;     //digit = 5
     sum = sum + digit;        //sum = 4 + 5
     num = num /10;            //num = 0
   }

   return sum;

}
```